

El siguiente código de una aplicación de gestión de **biblioteca virtual**.

El código ya está bien diseñado y hace uso de distintos **patrones de diseño creacionales y estructurales**.

Tarea: Identificar, en cada fragmento del código, qué patrón se aplica y justificar vuestra respuesta.

```
import java.util.*;

class Libro implements Cloneable {
    private String titulo;
    private String autor;
    private int paginas;

    public Libro(String titulo, String autor, int paginas) {
        this.titulo = titulo;
        this.autor = autor;
        this.paginas = paginas;
    }

    @Override
    public Libro clone() {
        try {
            return (Libro) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }

    @Override
    public String toString() {
        return titulo + " - " + autor + " (" + paginas + " páginas)";
    }
}

abstract class RecursoCrear {
    public abstract Libro crearLibro(String titulo, String autor, int paginas);
}
```

```

}

class LibroCrear extends RecursoCrear {
    @Override
    public Libro crearLibro(String titulo, String autor, int paginas) {
        return new Libro(titulo, autor, paginas);
    }
}

class Catalogo {
    private static Catalogo instancia;
    private List<Libro> libros = new ArrayList<>();

    private Catalogo() {}

    public static Catalogo getInstance() {
        if (instancia == null) {
            instancia = new Catalogo();
        }
        return instancia;
    }

    public void agregarLibro(Libro libro) {
        libros.add(libro);
    }

    public List<Libro> getLibros() {
        return libros;
    }
}

interface SistemaPago {
    void procesarPago(String usuario, double cantidad);
}

```

```
class SistemaPagoExterno {  
    public void makePayment(String name, double amount) {  
        System.out.println("Pago externo procesado para " + name + ": " +  
amount + "€");  
    }  
}
```

```
class PagoProd implements SistemaPago {  
    private SistemaPagoExterno externo;  
  
    public PagoProd(SistemaPagoExterno externo) {  
        this.externo = externo;  
    }  
  
    @Override  
    public void procesarPago(String usuario, double cantidad) {  
        externo.makePayment(usuario, cantidad);  
    }  
}
```

```
abstract class LibroExt extends Libro {  
    protected Libro libro;  
  
    public LibroExt(Libro libro) {  
        super(libro.toString(), "", 0); // valores ficticios  
        this.libro = libro;  
    }  
  
    @Override  
    public String toString() {  
        return libro.toString();  
    }  
}
```

```
class LibroConAudio extends LibroExt {  
    public LibroConAudio(Libro libro) {
```

```

        super(libro);
    }

    @Override
    public String toString() {
        return super.toString() + " [Incluye versión en audio]";
    }
}

class LibroConNotas extends LibroExt {
    public LibroConNotas(Libro libro) {
        super(libro);
    }

    @Override
    public String toString() {
        return super.toString() + " [Incluye notas del autor]";
    }
}

// ===== CLIENTE =====

public class BibliotecaApp {
    public static void main(String[] args) {
        RecursoFactory factory = new LibroFactory();
        Libro libro1 = factory.crearLibro("El Quijote", "Cervantes", 1000);
        Libro libro2 = factory.crearLibro("1984", "Orwell", 300);

        Libro copia = libro1.clone();

        Catalogo catalogo = Catalogo.getInstance();
        catalogo.agregarLibro(libro1);
        catalogo.agregarLibro(libro2);
        catalogo.agregarLibro(copia);

        Libro libroAudio = new LibroConAudio(new LibroConNotas(libro2));
    }
}

```

```
SistemaPago pago = new PagoProd(new SistemaPagoExterno());
pago.procesarPago("Ana", 9.99);

// Mostrar resultados
System.out.println("Catálogo de libros:");
catalogo.getLibros().forEach(System.out::println);

System.out.println("Libro especial decorado:");
System.out.println(libroAudio);
}
}
```